



Agent Platform — Documentation

Four AI agent applications and a shared MCP tool server, running on Cloudflare Workers. Architecture, operations, and troubleshooting for trainees.

Landing Page

`ai-agents-47w.pages.dev`

Trip Planner — LangGraph

`trip-planner-8xe.pages.dev`

Career Assistant

`career-assistant-3by.pages.dev`

Personal Assistant

`personal-assistant-8ve.pages.dev`

Trip Planner — Multi-Agent

`trip-planner-a2a.pages.dev`

MCP Search Server

`mcp-search-server.workers.dev`

Reference guide for the five deployed applications in this repository, the MCP server behind them, and the operational procedures that keep them running.

Audience: trainees using, extending, or supporting the platform.

Last updated: 2026-08-28.

Contents

1. [Platform Overview](#)
 2. [Technology Stack](#)
 3. [Application List & URLs](#)
 4. [How Each App Works \(Detailed\)](#)
 5. [Common Issues & Fixes](#)
 6. [Deployment Information](#)
 7. [Environment Variables](#)
 8. [Testing](#)
 9. [Troubleshooting Flowchart](#)
 10. [Frequently Asked Questions](#)
 11. [Contact & Support](#)
 12. [Appendix](#)
-

1. Platform Overview

What is this platform?

A set of four AI agent applications plus a landing page, built as the practical track of the AI Agent Engineering Course. Each application is a step up in agent architecture:

- a single agent calling one tool,
- a workflow of agents inside one process,
- the same workflow split into independent agent services that talk over a protocol,
- and the same patterns applied to a new domain.

Everything runs on Cloudflare's edge. No servers to manage, no containers, no databases to operate except a hosted vector index.

Who is it for?

- **Trainees** working through the course — read the code, run the apps, break them, fix them.
- **Anyone evaluating agent architectures** — the same trip-planning problem is solved twice, once as an in-process workflow (project 03) and once as distributed agents (project 04). Compare them side by side.
- **Operators** — sections 5 through 9 are written for whoever is on call when something returns a 500.

The applications

App	One line
Landing Page	Static index of the four applications, with a live hostname on each card.
Personal Assistant	Chat over your own PDFs (RAG), the live web (API), or tools discovered from an MCP server (MCP).
Trip Planner — LangGraph	Three agents inside one Worker: search and budget run in parallel, then an itinerary agent writes the plan.
Trip Planner — Multi-Agent	The same plan produced by four independent Workers speaking the A2A protocol over service bindings.
Career Assistant	Analyses a resume, researches the live job market, and reports the gap between them.

A sixth component, the **MCP Search Server**, has no UI. It is the shared tool server — three of the four apps reach web search through it.

2. Technology Stack

Layer	Technology	Where it is used
Compute	Cloudflare Workers	Every backend. Eight deployed Workers total.
Hosting	Cloudflare Pages	All five front ends (React + Vite builds, and one static HTML page).
Session storage	Cloudflare KV	<code>CHAT_HISTORY</code> (Personal Assistant, 7-day TTL), <code>SESSIONS</code> (Career Assistant, 10-minute TTL).
Object storage	Cloudflare R2	Optional <code>DOCUMENTS</code> binding on the Personal Assistant — keeps the original PDF. Ingestion works without it.
Vector database	Pinecone	Personal Assistant document embeddings, via the Pinecone REST API (<code>llama-text-embed-v2</code> , integrated embedding).
Web search	SerpAPI	<code>web_search</code> and <code>image_search</code> in the MCP server; <code>google_jobs</code> directly in the Career Assistant.
LLM	Workers AI	All inference. <code>@cf/meta/llama-3.3-70b-instruct-fp8-fast</code> for JSON/tool calling, <code>@cf/openai/gpt-oss-120b</code> for prose, <code>@cf/baai/bge-base-en-v1.5</code> for Career Assistant embeddings.
Agent framework	LangGraph (<code>@langchain/langgraph</code>)	Career Assistant Worker — a compiled <code>StateGraph</code> . Project 03 uses a hand-written equivalent for a smaller bundle.
Agent protocol	A2A (JSON-RPC 2.0)	Project 04 — agent cards at <code>/.well-known/agent.json</code> , tasks via <code>tasks/send</code> .
Tool protocol	MCP (<code>2025-06-18</code>)	The search server, consumed by projects 02, 03, and 04.
Front end	React 19 + Vite 7	All four app clients. The landing page is hand-written HTML/CSS.
Tests	Vitest	Unit tests in the Worker packages.
CLI	Wrangler 4	Deploys, secrets, logs.

Why Workers AI and not the Anthropic API

The deployed Workers run entirely on Workers AI — no external LLM key is needed in production. The `server/` directories in each project are the original Node implementations from the course and do use `ANTHROPIC_API_KEY` / Ollama. They are kept for reference and for the OCR path (see PA-17 in the test plan); they are not what is deployed.

3. Application List & URLs

App	Front end (Pages)	Backend (Worker)
Landing Page	https://ai-agents-47w.pages.dev	—
Personal Assistant	https://personal-assistant-8ve.pages.dev	<code>https://personal-assistant-api.alsheikharama.workers.dev</code>
Trip Planner (LangGraph)	https://trip-planner-8xe.pages.dev	<code>https://trip-planner-api.alsheikharama.workers.dev</code>
Trip Planner (Multi-Agent)	https://trip-planner-a2a.pages.dev	<code>https://a2a-orchestrator.alsheikharama.workers.dev</code>
Career Assistant	https://career-assistant-3by.pages.dev	<code>https://career-assistant-api.alsheikharama.workers.dev</code>
MCP Search Server	—	<code>https://mcp-search-server.alsheikharama.workers.dev</code>

The three A2A sub-agents (`a2a-search-agent` , `a2a-budget-agent` , `a2a-itinerary-agent`) are deployed with `workers_dev = false` . They have no public URL by design — only the orchestrator can reach them, through service bindings.

3.1 Landing Page

Purpose. One entry point to the four applications.

Key features

- Four cards, each with name, description, and the live hostname it points at.
- Static HTML — no framework, no build step.
- Responsive grid, keyboard reachable, skip-to-content link.

How it works. `00-landing/public/index.html` is served by Cloudflare Pages (project `ai-agents` , branch `main`). Nothing else.

Screenshot

Screenshot placeholder: landing page, four cards visible.

3.2 Personal Assistant

Purpose. Ask questions and get answers grounded in a real source — your documents, or the live web.

Key features

- **Three modes**, switchable per message: `rag`, `api`, `mcp`.
- **PDF upload** up to 25 MB, chunked and embedded into Pinecone.
- **Citations** — RAG answers name the file and page (`[report.pdf, p.4]`).
- **Arabic support** — RTL line reconstruction during PDF extraction, diacritic-insensitive matching, and a language rule that answers an Arabic question in Arabic.
- **Conversation memory** in KV, scoped per session and per mode.
- **Small-talk shortcut** — greetings skip tool calling entirely, saving a model round trip and quota.

How it works. The client posts to `/api/chat` with the message, a session id, and a mode. The Worker loads history from KV, picks the tool set for that mode, and runs a tool-calling loop against Workers AI (max 3 rounds, 2 calls per round). The answer and the exchange go back to KV.

Screenshot

Screenshot placeholder: chat view with a cited RAG answer.

3.3 Trip Planner (LangGraph)

Purpose. Turn a destination, a length, a budget, and a headcount into a day-by-day itinerary built on current prices.

Key features

- Three agents: **search**, **budget**, **itinerary**.
- Search and budget run **in parallel**; itinerary waits for both.
- **Live SSE stream** — the UI shows each agent starting and finishing while the plan is still being written.
- Search is grounded in real results pulled through the MCP server; the itinerary is emitted as schema-constrained JSON.

How it works. `GET /api/trip/stream` opens an SSE stream. The Worker fans out to the two independent agents with `Promise.allSettled`, then hands both results to the itinerary agent, which returns structured JSON validated against a per-day schema.

Screenshot

Screenshot placeholder: agent status lane plus a rendered 7-day itinerary.

3.4 Trip Planner (Multi-Agent / A2A)

Purpose. The same output as 3.3, produced by four separately deployed services — to show what changes when agents stop sharing a process.

Key features

- **Four Workers:** orchestrator, search agent, budget agent, itinerary agent.

- **Agent discovery** — the orchestrator fetches each agent's card from `/well-known/agent.json` before dispatching any work, and the UI shows the cards it found.
- **A2A tasks** over JSON-RPC 2.0 (`tasks/send`), each with its own task id and lifecycle events in the stream.
- **Service bindings** — agent-to-agent calls never leave Cloudflare's network, so the sub-agents need no public URL and no shared auth token between them.

How it works. `GET /api/a2a/stream` → discovery phase → parallel `tasks/send` to search and budget → sequential `tasks/send` to itinerary with both artifacts in the prompt → `result` frame.

Screenshot

Screenshot placeholder: discovery cards and the task event log.

3.5 Career Assistant

Purpose. Tell someone what stands between the resume they have and the role they want, in a named market.

Key features

- **Resume analysis** with retrieval over the resume itself, so claims stay tied to text the candidate actually wrote.
- **Live market research** from real job postings (SerpAPI `google_jobs`, up to 8 analysed).
- **Gap analysis** — missing skills, strengths, and an ordered set of recommendations.
- **Three-node LangGraph** with per-node progress streamed to the UI.
- **Two-step session** — the resume is POSTed once, then streamed by id, because EventSource cannot send a body.

How it works. `POST /api/career/start` stores `{resume, targetMarket, targetRole}` in KV under a UUID with a 10-minute TTL and returns the id. `GET /api/career/stream?sessionId=...` reads it, **deletes it immediately** (single use), and runs the graph: `resumeAnalyzer → marketResearcher → gapAnalyst`.

Screenshot

Screenshot placeholder: gap analysis with skills and recommendations.

4. How Each App Works (Detailed)

4.1 Personal Assistant

Source: `02-personal-assistant/worker/` · **Worker:** `personal-assistant-api`

INGESTION PIPELINE

```
PDF upload → size/type check → R2 (optional) → text extraction
           → RTL-aware line reconstruction → Arabic normalisation
           → chunking (1000 chars, 200 overlap)
           → batch embed (96 per batch, llama-text-embed-v2)
           → Pinecone upsert (id prefix = objectKey)
```

- `POST /api/ingest`, `multipart/form-data`, field `file`.
- Rejected: non-PDF (400), over 25 MB (413), no readable text layer (scanned/image-only PDFs — the Worker has no OCR path).
- Duplicate detection is by prefix listing; send `force=true` in the form to re-ingest anyway.
- Chunk metadata carries `source`, `pageNumber`, and `text`, which is what produces the `[file, p.N]` citation.

```
curl -X POST https://personal-assistant-api.alsheikharama.workers.dev/api/ingest \
-F "file=@handbook.pdf"
```

CHAT MODES

Mode	Tools	Behaviour
<code>rag</code>	<code>search_knowledge_base</code>	Must call the tool before answering, every time. Cites the filename and page. Never answers documents from model memory.
<code>api</code>	<code>web_search</code> , <code>image_search</code>	Calls SerpAPI directly from the Worker. For current events, live data, images.
<code>mcp</code>	discovered at runtime	Calls <code>tools/list</code> on the MCP server and uses whatever it advertises. Same capability as <code>api</code> , reached through the protocol instead of hard-coded.

```
curl -X POST https://personal-assistant-api.alsheikharama.workers.dev/api/chat \
-H "Content-Type: application/json" \
-d '{"message":"What does the handbook say about leave?","sessionId":"demo-1","mode":"rag"}'
```

Response: `{"answer":"...", "mode":"rag"}`.

MEMORY

KV key is `chat:{mode}:{sessionId}`, TTL 7 days. Switching mode starts a separate thread under the same session id — deliberate, so a RAG conversation is not polluted by web results.

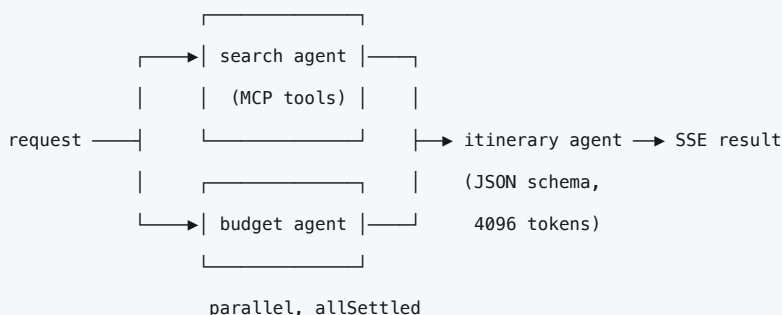
LOOP LIMITS

3 tool rounds, 2 calls per round, tool output truncated to 2500 characters, ~24k token context budget with a 1500-token margin. Hitting the round limit forces a synthesis pass rather than an empty answer.

4.2 Trip Planner (LangGraph)

Source: `03-trip-planner-langgraph/worker/` · **Worker:** `trip-planner-api`

WORKFLOW



- **Search agent** (`TEXT_MODEL`, `@cf/openai/gpt-oss-120b`) is required to call `web_search` before answering. It may call it more than once — attractions, hotels, flights. Prompt forbids filling gaps from memory.
- **Budget agent** (same prose model) splits the total budget across categories for the given days and headcount.
- **Itinerary agent** (`JSON_MODEL`, `@cf/meta/llama-3.3-70b-instruct-fp8-fast`) receives both outputs and emits JSON constrained to a per-day schema.

If either parallel agent rejects, the whole run fails with the collected messages — there is no partial render on this path.

MCP INTEGRATION

The Worker builds a JSON-RPC request against `MCP_SERVER_URL` and dispatches it through the **service binding** (`MCP`) when one is bound, or over plain HTTP when it is not. `MCP_SERVER_URL` is required in both cases. Either way the request carries `Authorization: Bearer $MCP_AUTH_TOKEN`.

STREAMING

```
curl -N "https://trip-planner-api.alsheikharama.workers.dev/api/trip/stream?
destination=Kyoto&days=5&budget=3000&people=2"
```

Frames:

```

data: {"type":"agent_status","agent":"search","status":"start"}
data: {"type":"agent_status","agent":"budget","status":"start"}
data: {"type":"agent_status","agent":"budget","status":"done"}
data: {"type":"agent_status","agent":"search","status":"done"}
data: {"type":"agent_status","agent":"itinerary","status":"start"}
data: {"type":"result","itinerary":{"...}}

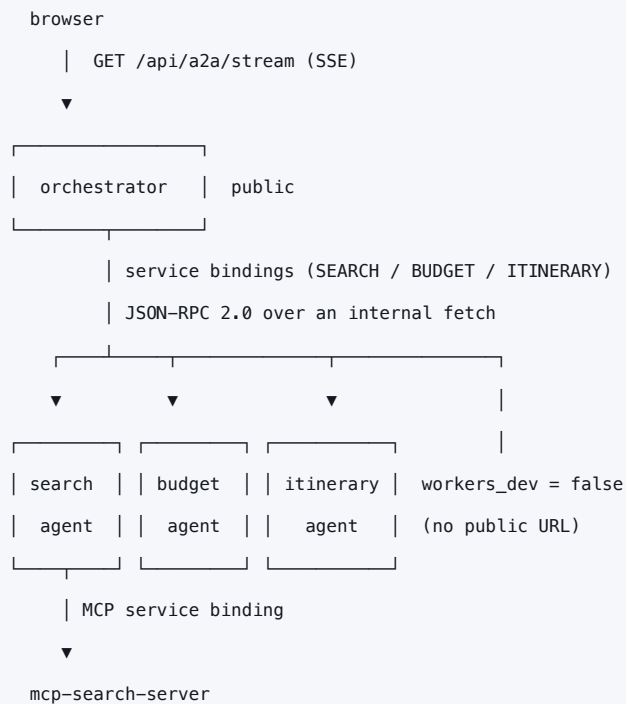
```

Errors arrive as `{"type":"error","message":"..."}` on the same stream, not as an HTTP status — a missing `destination` is the only case that 400s before the stream opens.

4.3 Trip Planner (Multi-Agent / A2A)

Source: `04-trip-planner-a2a/workers/` · **Workers:** `a2a-orchestrator`, `a2a-search-agent`, `a2a-budget-agent`, `a2a-itinerary-agent`

TOPOLOGY



THE A2A CONTRACT

Every agent Worker exposes the same two things:

1. `GET /.well-known/agent.json` — the agent card: name, description, version, capabilities, skills.
2. `POST /` — JSON-RPC 2.0, method `tasks/send`:

```
{
  "jsonrpc": "2.0",
  "id": "rpc-...",
  "method": "tasks/send",
  "params": {
    "id": "task-...",
    "message": { "role": "user", "parts": [{ "type": "text", "text": "..." }] }
  }
}
```

Reply:

```
{
  "jsonrpc": "2.0",
  "id": "rpc-...",
  "result": {
    "status": { "state": "completed" },
    "artifacts": [{ "name": "result", "parts": [{ "type": "text", "text": "..." }] }]
  }
}
```

Any method other than `tasks/send`, or a `jsonrpc` other than `"2.0"`, is rejected.

RUN PHASES

Phase	Frames emitted
Discovery	<code>phase:discovery</code> , then one <code>agent_discovered</code> per agent with its card
Parallel	<code>phase:parallel</code> , then <code>task_sent</code> / <code>task_done</code> for search and budget
Synthesis	<code>task_sent</code> / <code>task_done</code> for itinerary
Result	<code>result</code> with the parsed itinerary

WHY SERVICE BINDINGS

Sub-agent traffic stays inside Cloudflare — no public endpoint to secure, no egress, no extra latency. The trade-off is that you cannot `curl` an agent directly; to inspect one, use `wrangler tail` on it or deploy a preview with `workers_dev = true`.

4.4 Career Assistant

Source: `05-career-assistant/worker/` · **Worker:** `career-assistant-api`

THE GRAPH

```
START → resumeAnalyzer → marketResearcher → gapAnalyst → END
```

Compiled with `StateGraph` from `@langchain/langgraph`. Each node is wrapped so it emits `agent_status` start/done frames plus a free-text detail line (`Found 6 jobs` , and so on).

1. resumeAnalyzer — builds a retrieval index over the resume itself:

Index mode	When	How it retrieves
<code>embedding</code>	Resume is long enough to chunk and Workers AI returns usable vectors	Cosine similarity over <code>@cf/baai/bge-base-en-v1.5</code>
<code>keyword</code>	Embedding failed or returned an unusable shape	Token overlap scoring
<code>raw</code>	Resume too short to chunk	Whole text into the prompt

Everything is per request and in memory — no vector database on this path. Output is schema-constrained JSON: skills, experience, domain.

2. marketResearcher — builds a job query from role, market, and the detected domain, calls SerpAPI `google_jobs` , filters postings by domain, and analyses at most 8. Produces top skills, salary range, and demand signal.

3. gapAnalyst — takes both payloads and emits missing skills, existing strengths, and ordered recommendations.

TWO-STEP SESSION

```
SESSION=$(curl -s -X POST https://career-assistant-api.alsheikharama.workers.dev/api/career/start \
  -H "Content-Type: application/json" \
  -d '{"resume":"...", "targetMarket":"Germany", "targetRole":"Android Developer"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["sessionId"])')

curl -N "https://career-assistant-api.alsheikharama.workers.dev/api/career/stream?sessionId=$SESSION"
```

The session is deleted the moment the stream opens. Replaying the same `sessionId` returns `404 {"error":"Session not found"}` — by design. Unused sessions expire after 10 minutes.

5. Common Issues & Fixes

Issue	Cause	Fix
4006 from Workers AI	Daily Workers AI quota exhausted (10,000 neurons/day on the free plan)	Wait for midnight UTC, or move the account to Workers Paid (\$5/month)
MCP 401 Unauthorized	MCP_AUTH_TOKEN missing or different between the calling Worker and mcp-search-server	Set the same secret on all five Workers (see §6.3), then redeploy
"Cannot reach server" in the UI	Origin not on the CORS allowlist, or the Worker is down	curl the /api/health endpoint; check CLIENT_ORIGIN matches the Pages URL exactly, including scheme
Empty search results	SerpAPI query too narrow, or SERPAPI_API_KEY unset/out of credit	Retry with a broader query; verify the key with a direct SerpAPI call
Arabic text garbled	PDF extraction — glyph order or a missing text layer	Re-export the PDF with an embedded text layer; scanned PDFs need the OCR path in server/, which the Worker does not have
500 on ingest, "no readable text"	Image-only / scanned PDF	Same as above — the deployed Worker refuses it deliberately
Trip planner ends on error with no output	Search or budget agent rejected; the run discards both	Check wrangler tail; usually an MCP auth failure or a 4006 underneath
Career stream returns 404 immediately	sessionId already consumed, or older than 10 minutes	Call /api/career/start again — sessions are single use
Pinecone PINECONE_INDEX error	Secret/var not set on the Worker	npx wrangler secret put PINECONE_API_KEY; confirm PINECONE_INDEX

Known open defects

These are documented in testplan.md and are written to **fail**. A pass means someone fixed it and the test plan needs updating.

Case	What it documents
MCP-00	Fixed 2026-08-28. Auth used to fail <i>open</i> when MCP_AUTH_TOKEN was unset. <code>isAuthorized</code> now returns <code>false</code> and logs when the secret is missing, so a lost secret closes the endpoint instead of opening it
PA-04	RAG transcribes retrieved passages close to verbatim instead of summarising
PA-17	Scanned PDFs cannot be ingested in production — no OCR path in the Worker
TP-13	The response shape of @cf/openai/gpt-oss-120b is unverified; the helper silently handles both
A2A-10	A search-agent failure discards budget work that already completed
CA-08	Zero job postings still yield a confident market report

Case	What it documents
CA-09	Whether <code>google_jobs</code> returns results at all determines if CA-08 is an edge case or the normal path
Never clear <code>MCP_AUTH_TOKEN</code> on the deployed <code>mcp-search-server</code>. Since the MCP-09 fix a missing secret closes the endpoint rather than opening it, so clearing it breaks all four consuming Workers at once. Reproduce auth behaviour on a preview or throwaway Worker.	

6. Deployment Information

6.1 Prerequisites

- **Cloudflare account** with Workers, Pages, KV, and Workers AI enabled.
- **Node.js 18+** and npm.
- **Wrangler CLI** — `npm install -g wrangler`, or use `npx wrangler`.
- **Authenticated session** — `npx wrangler login`.
- **API keys:** Pinecone (Personal Assistant), SerpAPI (MCP server and Career Assistant).
- A **Pinecone index** using integrated embedding with `llama-text-embed-v2`.

Install everything first:

```
npm run install:all
```

6.2 Commands

Deploy order matters once: `mcp-search-server` first, because three Workers bind to it by service name.

```
# 1. Shared tool server – deploy this first
cd 01-mcp-search-server/worker && npm run deploy

# 2. Personal Assistant
cd 02-personal-assistant/worker && npm run deploy

# 3. Trip Planner (LangGraph)
cd 03-trip-planner-langgraph/worker && npm run deploy

# 4. Multi-agent – sub-agents before the orchestrator
cd 04-trip-planner-a2a/workers && npm run deploy:all

# 5. Career Assistant
cd 05-career-assistant/worker && npm run deploy
```

`deploy:all` runs search → budget → itinerary → orchestrator in that order. To deploy one agent:

```
cd 04-trip-planner-a2a/workers
npm run deploy:search      # or deploy:budget / deploy:itinerary / deploy:orchestrator
```

Front ends:

```

cd 02-personal-assistant/client && npm run build
npx wrangler pages deploy dist --project-name personal-assistant

cd 03-trip-planner-langgraph && npm --prefix client run build
npx wrangler pages deploy client/dist --project-name trip-planner

cd 04-trip-planner-a2a && npm --prefix client run build
npx wrangler pages deploy client/dist --project-name trip-planner-a2a --branch main

cd 05-career-assistant && npm --prefix client run build
npx wrangler pages deploy client/dist --project-name career-assistant

npx wrangler pages deploy 00-landing/public --project-name ai-agents --branch main

```

Vite reads `.env.production` at **build** time. Changing `VITE_API_URL` requires a rebuild, not just a redeploy.

Verify after every deploy:

```

for u in mcp-search-server/health \
    personal-assistant-api/api/health \
    trip-planner-api/api/health \
    a2a-orchestrator/api/health \
    career-assistant-api/api/health; do
    printf '%-45s %s\n' "$u" "$(curl -s -o /dev/null -w '%{http_code}%'
https://${u%/*}.alsheikharama.workers.dev/${u#*/})"
done

```

6.3 Secrets Management

Secrets are per Worker. Setting one does not propagate.


```
# MCP server
cd 01-mcp-search-server/worker
npx wrangler secret put SERPAPI_API_KEY
npx wrangler secret put MCP_AUTH_TOKEN

# Personal Assistant
cd 02-personal-assistant/worker
npx wrangler secret put PINECONE_API_KEY
npx wrangler secret put PINECONE_INDEX
npx wrangler secret put SERPAPI_API_KEY
npx wrangler secret put MCP_AUTH_TOKEN

# Trip Planner (LangGraph)
cd 03-trip-planner-langgraph/worker
npx wrangler secret put MCP_AUTH_TOKEN

# Multi-agent – only the search agent talks to MCP
cd 04-trip-planner-a2a/workers
npx wrangler secret put MCP_AUTH_TOKEN --config search-agent/wrangler.toml

# Career Assistant
cd 05-career-assistant/worker
npx wrangler secret put SERPAPI_API_KEY
```

Generate a token:

```
openssl rand -hex 32
```

The **same** `MCP_AUTH_TOKEN` value must be on `mcp-search-server`, `personal-assistant-api`, `trip-planner-api`, and `a2a-search-agent`. A mismatch shows up as a 401 from the tool call, which surfaces to the user as an agent that answers without searching or fails outright.

List what a Worker holds (names only, never values):

```
npx wrangler secret list
```

Local development uses `.dev.vars` in each Worker directory. Copy from `.dev.vars.example`. **Never commit it.**

7. Environment Variables

Legend: **var** = plain text in `wrangler.toml`, committed. **secret** = `wrangler secret put`, never in the repo.

01 — MCP Search Server (`mcp-search-server`)

Name	Kind	Description
<code>SERPAPI_API_KEY</code>	secret	SerpAPI key backing <code>web_search</code> and <code>image_search</code> . Without it, every tool call throws.
<code>MCP_AUTH_TOKEN</code>	secret	Bearer token required on <code>/mcp</code> . If unset, auth fails open (MCP-09).

02 — Personal Assistant (`personal-assistant-api`)

Name	Kind	Value / description
<code>AI_MODEL</code>	var	<code>@cf/meta/llama-3.3-70b-instruct-fp8-fast</code> — chat and tool calling
<code>CLIENT_ORIGIN</code>	var	<code>https://personal-assistant-8ve.pages.dev</code> — CORS allowlist
<code>CLIENT_ORIGIN_SUFFIXES</code>	var	<code>.personal-assistant-8ve.pages.dev</code> — allows Pages preview deploys
<code>PINECONE_NAMESPACE</code>	var	Pinecone namespace; empty means default
<code>MCP_SERVER_URL</code>	var	MCP endpoint. Always required — it is the request URL even when the <code>MCP</code> service binding carries the call; without a binding it is fetched over HTTP
<code>PINECONE_API_KEY</code>	secret	Pinecone API key
<code>PINECONE_INDEX</code>	secret	Pinecone index name
<code>SERPAPI_API_KEY</code>	secret	Powers <code>api</code> mode search directly from the Worker
<code>MCP_AUTH_TOKEN</code>	secret	Bearer token for <code>mcp</code> mode
<code>AI</code>	binding	Workers AI
<code>CHAT_HISTORY</code>	binding	KV — conversation history, 7-day TTL
<code>MCP</code>	binding	Service binding → <code>mcp-search-server</code>
<code>DOCUMENTS</code>	binding	R2, optional — stores the original PDF; ingestion works without it
<code>RATE_LIMITER</code>	binding	Rate limiting, 20 requests per 60 seconds keyed on <code>cf-connecting-ip</code> ; the middleware is skipped if the binding is ever removed

03 — Trip Planner LangGraph (`trip-planner-api`)

Name	Kind	Value / description
<code>TEXT_MODEL</code>	var	<code>@cf/openai/gpt-oss-120b</code> — search and budget prose
<code>JSON_MODEL</code>	var	<code>@cf/meta/llama-3.3-70b-instruct-fp8-fast</code> — itinerary JSON

Name	Kind	Value / description
<code>MCP_SERVER_URL</code>	var	MCP endpoint. Always required — the request URL for both the service binding and the HTTP path
<code>CLIENT_ORIGIN</code>	var	<code>http://localhost:5174,https://trip-planner-8xe.pages.dev</code>
<code>CLIENT_ORIGIN_SUFFIXES</code>	var	<code>.trip-planner-8xe.pages.dev</code>
<code>MCP_AUTH_TOKEN</code>	secret	Bearer token for the MCP server
<code>AI</code> , <code>MCP</code>	bindings	Workers AI; service binding → <code>mcp-search-server</code>
<code>RATE_LIMITER</code>	binding	Rate limiting, 20 requests per 60 seconds keyed on <code>cf-connecting-ip</code>

04 — Multi-Agent (four Workers)

`a2a-orchestrator`

Name	Kind	Value / description
<code>CLIENT_ORIGIN</code>	var	<code>http://localhost:5175,https://trip-planner-a2a.pages.dev</code>
<code>CLIENT_ORIGIN_SUFFIXES</code>	var	<code>.trip-planner-a2a.pages.dev</code>
<code>SEARCH</code> , <code>BUDGET</code> , <code>ITINERARY</code>	bindings	Service bindings to the three agents
<code>RATE_LIMITER</code>	binding	Rate limiting, 20 requests per 60 seconds keyed on <code>cf-connecting-ip</code>

`a2a-search-agent`

Name	Kind	Value / description
<code>AGENT_URL</code>	var	<code>binding://a2a-search-agent</code> — identity in the agent card
<code>TEXT_MODEL</code>	var	<code>@cf/openai/gpt-oss-120b</code>
<code>MCP_SERVER_URL</code>	var	MCP endpoint. Always required — see project 02
<code>MCP_AUTH_TOKEN</code>	secret	Bearer token for the MCP server
<code>AI</code> , <code>MCP</code>	bindings	Workers AI; service binding → <code>mcp-search-server</code>
<code>RATE_LIMITER</code>	binding	Rate limiting, 20 requests per 60 seconds keyed on <code>cf-connecting-ip</code>

`a2a-budget-agent` — `AGENT_URL` (`binding://a2a-budget-agent`), `TEXT_MODEL` (`@cf/openai/gpt-oss-120b`), `AI`.

`a2a-itinerary-agent` — `AGENT_URL` (`binding://a2a-itinerary-agent`), `JSON_MODEL` (`@cf/meta/llama-3.3-70b-instruct-fp8-fast`), `AI`.

05 — Career Assistant (`career-assistant-api`)

Name	Kind	Value / description
<code>AI_MODEL</code>	var	<code>@cf/meta/llama-3.3-70b-instruct-fp8-fast</code>
<code>CLIENT_ORIGIN</code>	var	<code>http://localhost:5175,https://career-assistant-3by.pages.dev</code>
<code>CLIENT_ORIGIN_SUFFIXES</code>	var	<code>.career-assistant-3by.pages.dev</code>
<code>EMBEDDING_MODEL</code>	var, optional	Overrides <code>@cf/baai/bge-base-en-v1.5</code>
<code>SERPAPI_API_KEY</code>	secret	<code>google_jobs</code> market research; the market node throws without it
<code>AI</code>	binding	Workers AI
<code>SESSIONS</code>	binding	KV — resume sessions, 600-second TTL, single use
<code>RATE_LIMITER</code>	binding	Rate limiting, 20 requests per 60 seconds keyed on <code>cf-connecting-ip</code>

Front ends (build time)

Name	Where	Value
<code>VITE_API_URL</code>	<code>client/.env.production</code>	The Worker URL for that app
<code>VITE_API_URL</code>	<code>client/.env.development</code>	<code>http://localhost:8787</code>

8. Testing

Manual acceptance suite

`testplan.md` — **75 cases** across the five apps, the MCP server, and the landing page. Every step is written against the live production deployment, not a local dev server.

Suite	Cases	Prefix
Landing Page	5	LP-01 → LP-05
MCP Search Server	9	MCP-01 → MCP-09
Personal Assistant	22	PA-01 → PA-22
Trip Planner (LangGraph)	13	TP-01 → TP-13
Trip Planner (Multi-Agent)	10	A2A-01 → A2A-10
Career Assistant	12	CA-01 → CA-12
Cross-cutting	4	X-01 → X-04

Case types: **Smoke, Functional, Negative, Security, Known issue, UX / A11y**. Run the smoke cases first — a failure there invalidates the rest of the run.

BEFORE YOU START

- Browser with DevTools open — Network and Console. Several cases read SSE frames from the EventStream tab.
- `curl` — every API-level case is a one-liner.
- The MCP bearer token in hand (MCP-03 → MCP-07). It exists only as a Cloudflare secret, in no local file.
- Four test files: a small text-layer PDF, the same PDF again, a non-PDF, a PDF over 25 MB, and a scanned PDF with no text layer.
- A resume in plain text — the Career Assistant takes pasted text, not a file.
- Run order matters twice: **PA-12 needs PA-11 first, CA-05 needs CA-04 first**. Everything else is independent.

Automated unit tests

```
cd 02-personal-assistant/worker && npm test      # agent, cors, ingest, intent, arabic
cd 03-trip-planner-langgraph/worker && npm test   # trip-agent, cors
cd 04-trip-planner-a2a/workers && npm test
cd 05-career-assistant/worker && npm test          # career-agent, cors, normalize, resume-rag
cd 02-personal-assistant/client && npm test        # sanitize
```

Watch mode: `npm run test:watch`. These run offline against mocked bindings — no Cloudflare account or quota needed.

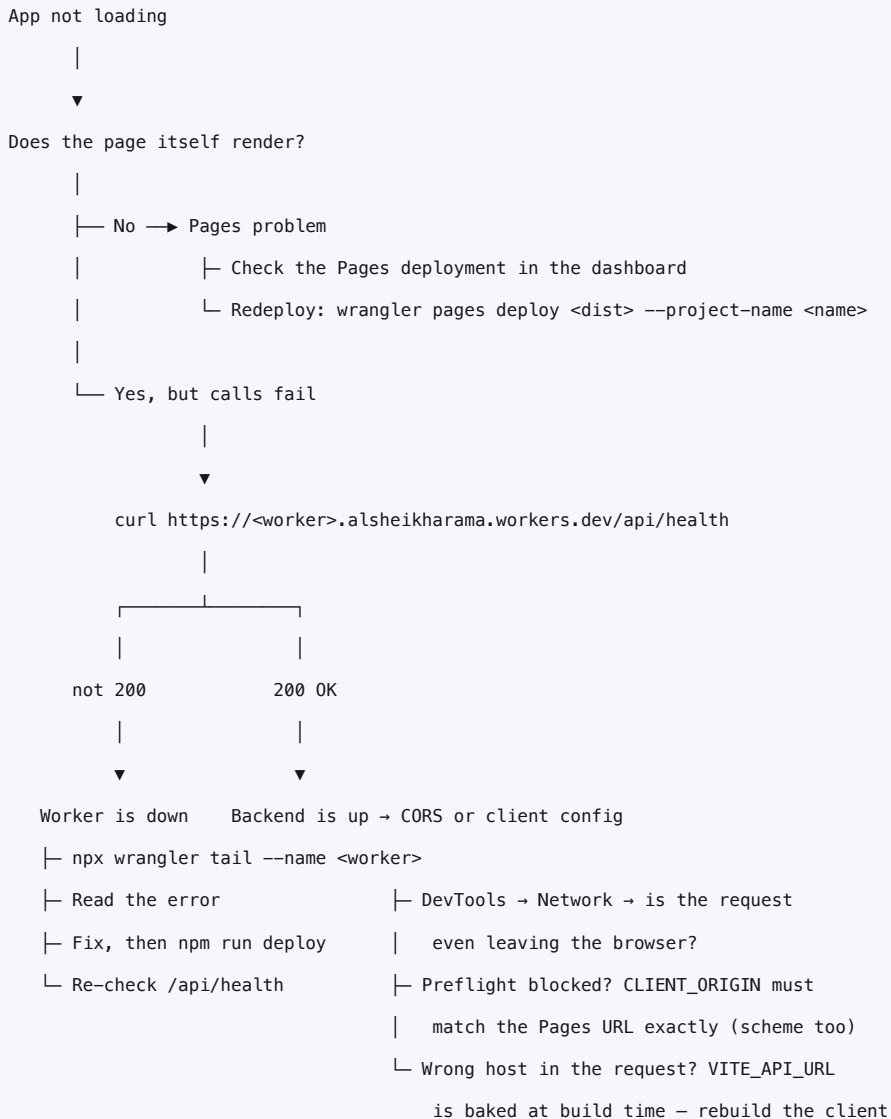
Smoke test in one command

```
for h in mcp-search-server/health personal-assistant-api/api/health \
    trip-planner-api/api/health a2a-orchestrator/api/health \
    career-assistant-api/api/health; do
    echo "$h → ${curl -s https://${h%/*}.alsheikharama.workers.dev/${h#*/}}"
done
```

All five must return `{"ok":true...}`.

9. Troubleshooting Flowchart

"App not loading"



"Search not working"

Search returns nothing / the agent answers from memory

|

▼

Is the MCP server up?

```
curl https://mcp-search-server.alsheikharama.workers.dev/health
```

|

|— not ok → Redeploy the MCP server, then re-check

|

▼ ok

Does an authenticated tools/list succeed?

```
curl -X POST .../mcp -H "Authorization: Bearer $MCP_AUTH_TOKEN" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
```

|

|— 401 → Token mismatch

|

└ Re-put the SAME MCP_AUTH_TOKEN on the MCP server

|

AND on personal-assistant-api, trip-planner-api,

|

a2a-search-agent. Redeploy each.

|

▼ 200

Does tools/call return results?

|

|— "Tool failed: SERPAPI_API_KEY is not set" → set the secret

|

|— SerpAPI 401 / 429 → key invalid or out of credit – check the SerpAPI dashboard

|

|— "No results found." → query too narrow; retry broader

|

▼ results come back

Problem is in the calling Worker

| npx wrangler tail --name <worker>

|— Look for 4006 → daily AI quota exhausted (\$10)

|— Look for a tool-round cap – 3 rounds, 2 calls per round

Reading logs

```
npx wrangler tail --name personal-assistant-api
```

```
npx wrangler tail --name trip-planner-api
```

```
npx wrangler tail --name a2a-search-agent          # sub-agents have no public URL – tail is the way in
```

```
npx wrangler tail --name career-assistant-api
```

```
npx wrangler tail --name mcp-search-server
```

Observability is enabled on every Worker, so logs are also in the Cloudflare dashboard under **Workers & Pages** → <worker> → **Logs**.

10. Frequently Asked Questions

Q: Why am I getting a `4006` error? A: The daily Workers AI quota is exhausted — 10,000 neurons per day on the free plan. Wait for the reset at midnight UTC, or upgrade to Workers Paid (\$5/month). Every app on the platform shares that one account quota, so a heavy trip-planning session can starve the Personal Assistant.

Q: How do I rotate `MCP_AUTH_TOKEN`? A: Generate one value and put it on all four consumers plus the server, then redeploy:

```
TOKEN=$(openssl rand -hex 32)
echo "$TOKEN" # keep it somewhere safe – you cannot read it back out of Wrangler

cd 01-mcp-search-server/worker      && echo "$TOKEN" | npx wrangler secret put MCP_AUTH_TOKEN
cd 02-personal-assistant/worker     && echo "$TOKEN" | npx wrangler secret put MCP_AUTH_TOKEN
cd 03-trip-planner-langgraph/worker && echo "$TOKEN" | npx wrangler secret put MCP_AUTH_TOKEN
cd 04-trip-planner-a2a/workers      && echo "$TOKEN" | npx wrangler secret put MCP_AUTH_TOKEN --config
search-agent/wrangler.toml
```

Do the MCP server **last** if you cannot tolerate a gap — consumers with the new token get 401s until the server has it too. Verify with MCP-02 on all four consumers afterwards.

Q: Can I use this in production? A: Yes, on the Workers Paid plan (\$5/month) — that lifts the AI quota and the request limits. Every public Worker now carries a rate limiter (20 requests per 60 seconds per IP) and MCP-09 is fixed, so the two blockers are cleared. Keep the CORS allowlist tight, and read the remaining open defects in §5 before you rely on the market or itinerary payloads.

Rate limiting on Workers is enforced per Cloudflare location and is eventually consistent, not a global counter. It is a quota guard against runaway or abusive traffic, not an authentication control. A client that opens a fresh connection per request may see its first few requests spread across instances before the limit engages.

Q: How do I reset the Pinecone index? A: Delete the vectors from the Pinecone console, or delete and recreate the index with the same name and `llama-text-embed-v2` integrated embedding. The Worker reads `PINECONE_INDEX` and looks the host up on each cold start, so a recreated index of the same name needs no redeploy. Then re-upload the PDFs through `/api/ingest`.

Q: Which app do I run to see MCP working? A: Personal Assistant in `mcp` mode. It calls `tools/list` at request time and uses whatever the server advertises — add a tool to the MCP server and it appears without touching the assistant.

Q: Why are the three A2A agents not reachable by URL? A: `workers_dev = false`. They exist only behind the orchestrator's service bindings. To inspect one, `wrangler tail` it, or deploy a preview with `workers_dev = true`.

Q: Why does the Career Assistant need two requests? A: `EventSource` cannot send a request body, and a resume is too large for a query string. So the resume is POSTed to `/api/career/start`, held in KV for 10 minutes, and the stream fetches it by id.

Q: My Arabic PDF comes out scrambled. Why? A: The Worker reconstructs RTL lines from glyph positions, which works on PDFs with a proper text layer. If the PDF was scanned, or its text layer is malformed, there is nothing to reconstruct — re-export it from the source document. The Worker has no OCR path (PA-17).

Q: Which model does what? A: `@cf/openai/gpt-oss-120b` for prose (search briefings, budget narratives). `@cf/meta/llama-3.3-70b-instruct-fp8-fast` for tool calling and schema-constrained JSON. `@cf/baai/bge-base-en-v1.5` for Career Assistant embeddings; `llama-text-embed-v2` inside Pinecone for documents.

Q: Can I run all of this locally? A: The Workers, yes — `npm run dev` in any `worker/` directory starts `wrangler dev` on port 8787, with secrets read from `.dev.vars`. Service bindings do not resolve against deployed Workers in local dev, so a locally-run Worker reaches the deployed MCP server over HTTP at `MCP_SERVER_URL`.

Q: What are the `server/` directories? A: The original Node/Express implementations from the course, kept for reference. They use Anthropic or Ollama and are not what is deployed. Production is `worker/` (and `workers/` in project 04) everywhere.

11. Contact & Support

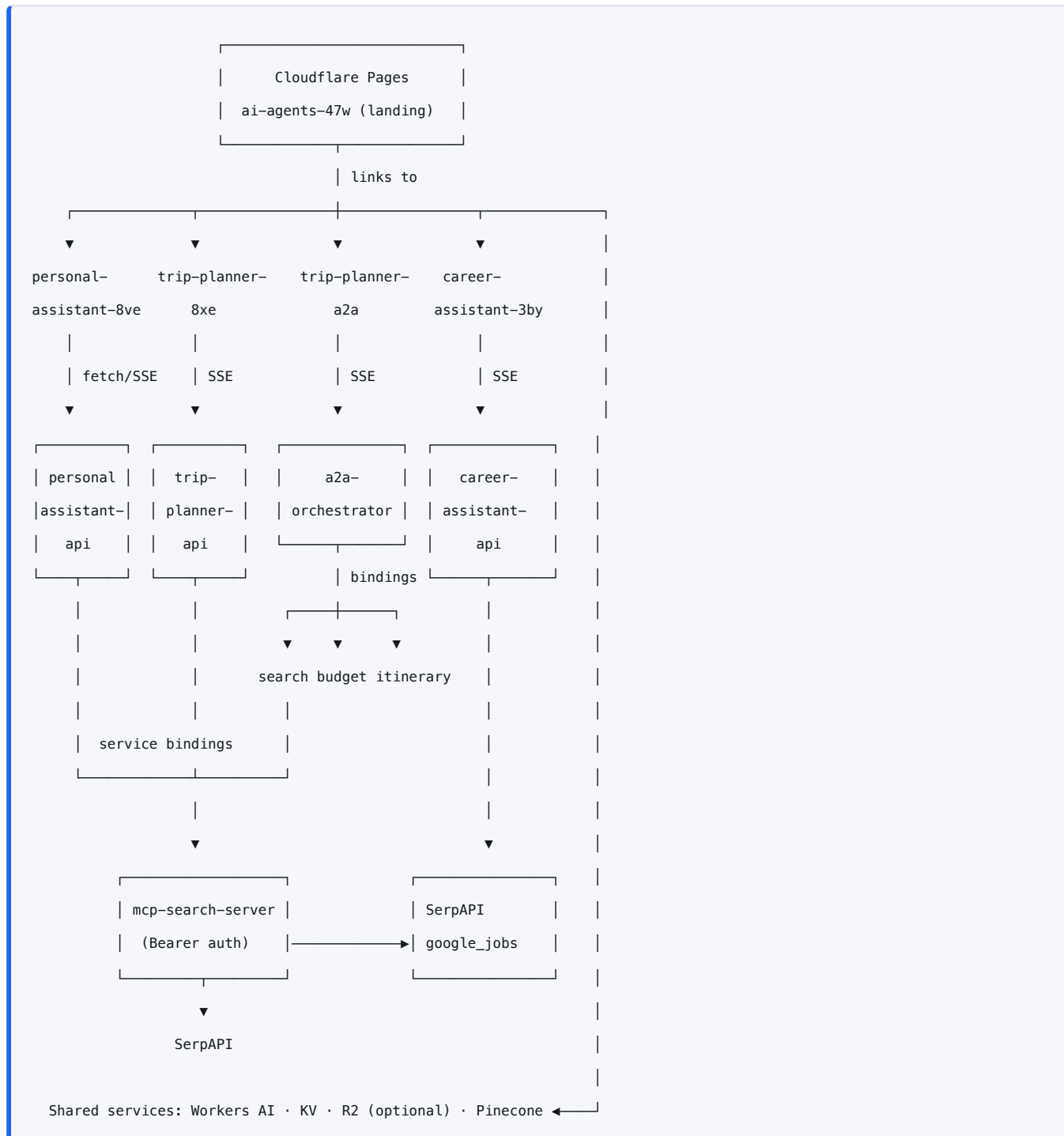
Resource	Where
This document	<code>PROJECT_DOCUMENTATION.md</code> (repository root)
Test plan	<code>testplan.md</code> — 75 acceptance cases
Repository	https://github.com/ramaalsheikha/ai-agents-course
Cloudflare Dashboard	https://dash.cloudflare.com/
Per-project notes	<code>0X-<project>/README.md</code> , plus <code>02-personal-assistant/DEPLOY.md</code>
Course	https://tariqlabs.com/courses/ai-agents/
Workers AI models	https://developers.cloudflare.com/workers-ai/models/
MCP specification	https://modelcontextprotocol.io
SerpAPI dashboard	https://serpapi.com/dashboard — key status and remaining credit
Pinecone console	https://app.pinecone.io — index health and vector count

Raising a problem: include the app name, the Worker name, the UTC timestamp, and the relevant `wrangler tail` lines. Never paste a secret value into an issue.

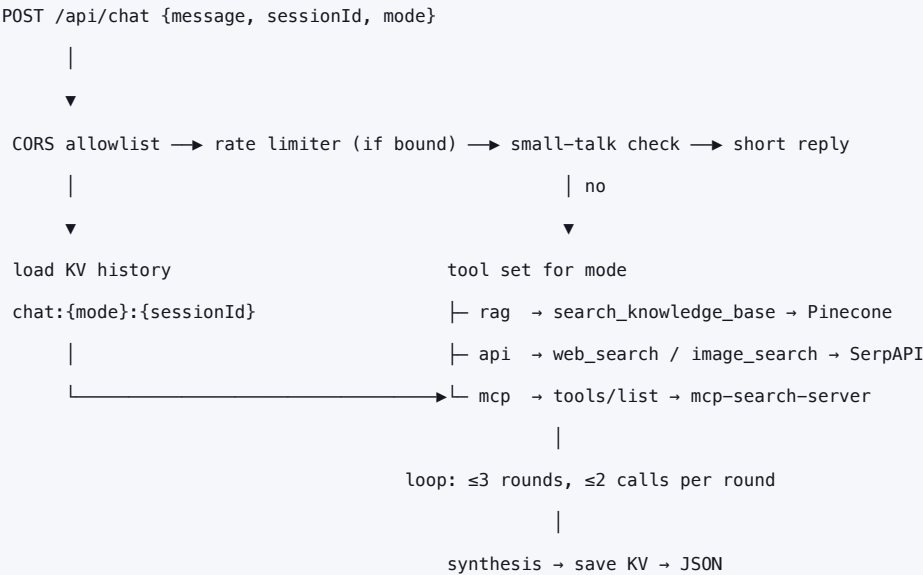
12. Appendix

12.1 Architecture diagrams

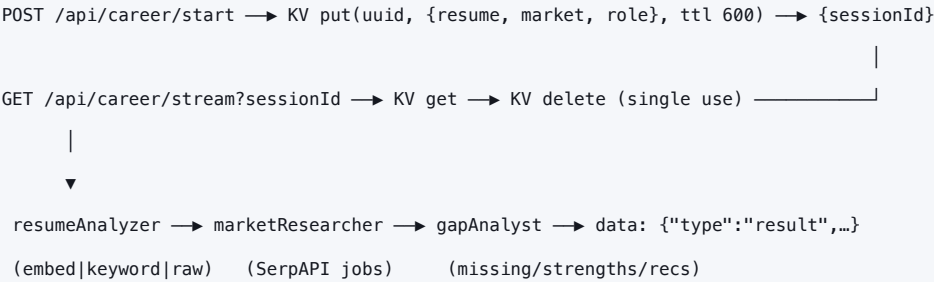
Platform



Personal Assistant — request path



Career Assistant — two-step session



12.2 API reference

MCP-SEARCH-SERVER			
Method	Path	Auth	Notes
GET	/health	none	{"ok":true,"server":"serp-search-mcp"}
POST	/mcp	Bearer \$MCP_AUTH_TOKEN	JSON-RPC 2.0, protocol 2025-06-18

Methods: initialize , ping , tools/list , tools/call . Tools: web_search , image_search (both take {query} , return the top 5 results as markdown text). Anything else 404s; non-POST on /mcp is 405.

```
curl -X POST https://mcp-search-server.alsheikhrama.workers.dev/mcp \
-H "Authorization: Bearer $MCP_AUTH_TOKEN" \
-H "Content-Type: application/json" \
-d '{"jsonrpc":"2.0","id":1,"method":"tools/call",
  "params":{"name":"web_search","arguments":{"query":"cloudflare workers ai pricing"}}}'
```

PERSONAL-ASSISTANT-API

Method	Path	Body	Returns
GET	/api/health	—	{"ok":true}
POST	/api/chat	{message, sessionId?, mode?}	{answer, mode}
POST	/api/ingest	multipart: file, force?	{ok, objectKey, ...}

Errors: 400 missing message / non-PDF, 413 over 25 MB, 429 rate limited, 500 with {error}.

TRIP-PLANNER-API

Method	Path	Query	Returns
GET	/api/health	—	{"ok":true}
GET	/api/trip/stream	destination (required), days (7), budget (2000), people (2)	SSE

SSE frame types: agent_status, result, error.

A2A-ORCHESTRATOR

Method	Path	Query	Returns
GET	/api/health	—	{"ok":true}
GET	/api/a2a/stream	same as above	SSE

SSE frame types: phase, agent_discovered, task_sent, task_done, result, error.

AGENT WORKERS (INTERNAL ONLY)

Method	Path	Returns
GET	/.well-known/agent.json	Agent card
POST	/	JSON-RPC 2.0 tasks/send → {status:{state:"completed"}, artifacts:[...]}

CAREER-ASSISTANT-API

Method	Path	Body / Query	Returns
GET	/api/health	—	{"ok":true}
POST	/api/career/start	{resume, targetMarket, targetRole}	{sessionId}
GET	/api/career/stream	sessionId	SSE

Errors: 400 missing field, 404 {"error":"Session not found"} for an unknown, expired, or already-consumed session.

SSE frame types: `agent_status` (with a `detail` line), `result` (`resumeAnalysis`, `marketResearch`, `gapAnalysis` as JSON strings), `error`.

12.3 Limits and defaults

Limit	Value	Where
Upload size	25 MB	Personal Assistant ingest
Chunk size / overlap	1000 / 200 chars	Personal Assistant ingest
Embedding batch	96 chunks	Personal Assistant ingest
Tool rounds / calls per round	3 / 2	Personal Assistant, Trip Planner, A2A search
Tool result truncation	2500 chars	same
Model context budget	~24,000 tokens (1500 margin)	Personal Assistant
Max output tokens	1024 search · 768 budget · 4096 itinerary	Trip Planner
Chat history TTL	7 days	KV <code>CHAT_HISTORY</code>
Career session TTL	600 seconds, single use	KV <code>SESSIONS</code>
Job postings analysed	8	Career Assistant
Rate limit	20 requests / 60 s per IP	every public Worker
Free Workers AI quota	10,000 neurons/day	account-wide

12.4 Version history

Version	Date	Change
1.1	2026-08-28	MCP-09 fixed (auth now fails closed); rate limiting added to all four public Workers; Cloudflare account id redacted from the session notes ahead of the repository going public
1.0	2026-08-28	First release of this document — covers all five apps, the MCP server, and operations
—	2026-08-26	<code>testplan.md</code> added — 75 acceptance cases
—	2026-08	Career Assistant grounded in retrieved resume text and real postings; partial results render instead of crashing
—	2026-08	OpenAI-shaped Workers AI replies handled across the prose model paths
—	2026-08	Light theme rolled out across all five front ends